

Lab18 – Service types and Ingress

Service

In this exercise, you will create a Deployment and expose a container port for its Pods. You will demonstrate the differences between the service types ClusterIP and NodePort.

1. Create a Service named `myapp` of type `ClusterIP` that exposes port 80 and maps to the target port 80.

```
vagrant@kubernetes-control-plane:~/lab10$ kubectl create service clusterip myapp --tcp=80:80
service/myapp created
vagrant@kubernetes-control-plane:~/lab10$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP    23m
myapp        ClusterIP   10.97.234.12 <none>        80/TCP     12s
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$
```

2. Create a Deployment named `myapp` that creates 1 replica running the image `nginx:1.23.4-alpine`. Expose the container port 80.

```
vagrant@kubernetes-control-plane:~/lab10$ kubectl create deployment myapp --image=nginx:1.23.4-alpine --port=80
deployment.apps/myapp created
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$ kubectl get deployments,pods
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/myapp               0/1     1             0           9s

NAME                                READY   STATUS              RESTARTS   AGE
pod/myapp-64fb67cd9-r8f8l           0/1     ContainerCreating   0           9s
pod/pod1                             1/1     Running             0          20m
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$
```

3. Scale the Deployment to 2 replicas.

```
vagrant@kubernetes-control-plane:~/lab10$ kubectl scale deployment myapp --replicas=2
deployment.apps/myapp scaled
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$ kubectl get deployments,pods
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/myapp               2/2     2             2          24m

NAME                                READY   STATUS              RESTARTS   AGE
pod/myapp-64fb67cd9-hsbk5           1/1     Running             0          12s
pod/myapp-64fb67cd9-r8f8l           1/1     Running             0          24m
pod/pod1                             1/1     Running             0          44m
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$
```

4. Create a temporary Pod using the image `busybox` and run a `wget` command against the IP of the service.

Determine the cluster IP and the port for the Service. In this case, it's `10.97.234.12:80`. Alternatively, you can use the DNS name `myapp`. Use the information with the `wget` command.

```
vagrant@kubernetes-control-plane:~/lab10$ kubectl run tmp --image=busybox --restart=Never -it --rm -- wget -O- 10.97.234.12:80
Connecting to 10.97.234.12:80 (10.97.234.12:80)
writing to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

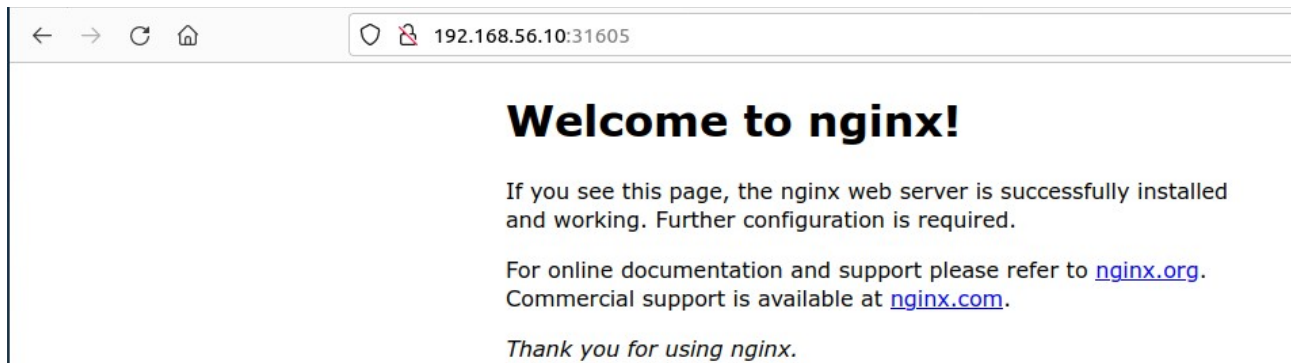
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
- 100% |*****| 615 0:00:00 ETA
written to stdout
pod "tmp" deleted
vagrant@kubernetes-control-plane:~/lab10$
```

5. Change the service type so that the Pods can be reached from outside of the cluster. You must turn the type of the service into `NodePort` to expose it outside of the cluster. Now, the service should expose a port in the 30000 range.

```
ports:
- name: 80-80
  port: 80
  protocol: TCP
  targetPort: 80
selector:
  app: myapp
sessionAffinity: None
type: NodePort
status:
```

```
vagrant@kubernetes-control-plane:~/lab10$ kubectl edit service myapp
service/myapp edited
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP          58m
myapp        NodePort    10.97.234.12 <none>        80:31605/TCP     35m
vagrant@kubernetes-control-plane:~/lab10$
vagrant@kubernetes-control-plane:~/lab10$
```

6. Run a browser or `wget` command against the service from outside of the cluster.



Ingress and Ingress Controller

In this exercise, you will create an Ingress with a simple rule that routes traffic to a Service.

The Kubernetes documentation

(<https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>) lists a wide range of Ingress Controller implementations.

7. Use command bellow to implement an ingress controller.

Kubectl apply -f <https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.1.1/deploy/static/provider/cloud/deploy.yaml>

```
vagrant@k8s-control-plane:~$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.1.1/deploy/static/provider/cloud/deploy.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
configmap/ingress-nginx-controller created
clusterrole.rbac.authorization.k8s.io/ingress-nginx created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx created
role.rbac.authorization.k8s.io/ingress-nginx created
rolebinding.rbac.authorization.k8s.io/ingress-nginx created
service/ingress-nginx-controller-admission created
service/ingress-nginx-controller created
deployment.apps/ingress-nginx-controller created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-admission created
serviceaccount/ingress-nginx-admission created
clusterrole.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
role.rbac.authorization.k8s.io/ingress-nginx-admission created
rolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
vagrant@k8s-control-plane:~$
```

8. The Ingress controller will run as a Pod in the `ingress-nginx` namespace. Make sure that the Pod `ingress-nginx-controller-...` transitions into the "Running" status.

```
vagrant@kubernetes-control-plane:~$ kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-pzlrh 0/1     Completed 0           6m
ingress-nginx-admission-patch-xf9zl   0/1     Completed 2           6m
ingress-nginx-controller-f7f5995cc-gfbbd 1/1     Running   0           6m
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$
```

Using Ingress

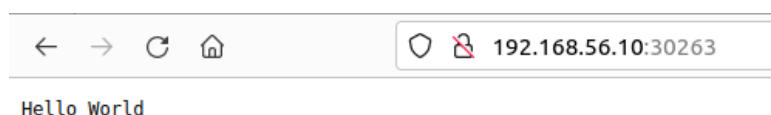
9. Create a new Deployment named `web` that controls a single replica running the image `bmuschko/nodejs-hello-world:1.0.0` on port 3000.

```
vagrant@kubernetes-control-plane:~$ kubectl create deployment web --image=bmuschko/nodejs-hello-world:1.0.0
deployment.apps/web created
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$ kubectl get deployment web
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
web     1/1     1            1           65s
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$
```

10. Expose the Deployment with a Service named `web` of type `NodePort`. The Service routes traffic to the Pods controlled by the Deployment `web`.

```
vagrant@kubernetes-control-plane:~$ kubectl expose deployment web --type=NodePort --port=3000
service/web exposed
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$ kubectl get service web
NAME    TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
web     NodePort    10.99.70.50   <none>        3000:30263/TCP 10s
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$
```

11. Make a request to the endpoint of the application on the context path `/`. You should see the message "Hello World".



12. Create an Ingress that exposes the path `/` for the host `hello-world.exposed`. The traffic should be routed to the Service created earlier.

```

vagrant@k8s-control-plane:~$ vim ingress.yaml
vagrant@k8s-control-plane:~$ cat ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1
spec:
  rules:
    - host: hello-world.exposed
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 3000

vagrant@k8s-control-plane:~$
vagrant@k8s-control-plane:~$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/hello-world-ingress created
vagrant@k8s-control-plane:~$

```

13. List the Ingress object. The value for the IP address will populate after waiting for a little while. You may have to run the command multiple times.

```

vagrant@k8s-control-plane:~$ kubectl get ingress hello-world-ingress
NAME                CLASS      HOSTS                ADDRESS      PORTS      AGE
hello-world-ingress <none>      hello-world.exposed  80           10m
vagrant@k8s-control-plane:~$

```

14. Add an entry in `/etc/hosts` that maps the virtual node IP address to the host `hello-world.exposed`.

```

brahim@Training:~/k8s-lab$ sudo vim /etc/hosts
brahim@Training:~/k8s-lab$ cat /etc/hosts
127.0.0.1      Training
127.0.0.1      localhost
192.168.56.10  hello-world.exposed
192.168.205.1  Training      Training
brahim@Training:~/k8s-lab$
brahim@Training:~/k8s-lab$

```

15. Make a request to `http://hello-world.exposed`. You should see the message "Hello World".

